

Empirical Pareto Frontier of Model Compression Paradigms for Ultra-Low-Resource TinyML

Narendra Satish

Abstract—Deploying deep learning on ultra-low-power microcontrollers (TinyML) requires aggressive model compression to fit strict SRAM (≤ 64 KB) and Flash (≤ 256 KB) budgets. However, the interactions among post-training integer quantization (PTQ), input feature reduction, magnitude-based weight pruning, and knowledge distillation remain insufficiently characterized under sub-4KB storage and sub-400 multiply-accumulate (MAC) constraints. In this paper, we present an empirical 4D Pareto characterization and low-level FlatBuffer artifact benchmark of 12 serialized TensorFlow Lite (TFLite) models spanning these four compression paradigms evaluated on 55,998 multi-sensor engine diagnostic records. We extract the four-objective empirical Pareto frontier across Test Accuracy, Model Size, Theoretical Active MACs, and Empirical Host Inference Latency. Our empirical findings demonstrate that: (1) knowledge distillation via a compact student architecture (student_b_16_4_fp32) establishes the highest test accuracy (75.14%) while reducing storage by 7.9% (3,584 B) over the uncompressed baseline; (2) 75% magnitude pruning achieves the lowest computational demand (96 theoretical active MACs, a 75% reduction) while retaining 74.82% accuracy, but exhibits computational sparsity without demonstrated storage compression in standard FlatBuffer formats (3,920 B vs. 3,892 B); and (3) full integer INT8 quantization achieves zero-floating-point execution graphs (0 float32 tensors) with minimal accuracy variation (-0.04% to $+0.44\%$). We conclude with an architectural characterization of the six Pareto-optimal models, offering rigorous empirical baselines for edge AI engineers navigating memory-compute-accuracy trade-offs.

Index Terms—TinyML, Model Compression, Pareto Optimization, Integer Quantization, Structured Pruning, Knowledge Distillation, Embedded Machine Learning.

I. Introduction

THE rapid emergence of Edge Artificial Intelligence and Tiny Machine Learning (TinyML) has enabled real-time inference directly on resource-constrained microcontrollers (MCUs) at the extreme edge [1]–[3]. In high-frequency sensor applications such as automotive powertrain monitoring, industrial vibration analysis, and structural health diagnostics, performing on-device classification eliminates latency penalties, communication bandwidth bottlenecks, and privacy vulnerabilities associated with cloud offloading [4]–[6].

However, microcontrollers operate under severe hardware constraints: standard embedded devices frequently offer fewer than 256KB of on-chip non-volatile Flash storage and as little as 32KB to 64KB of static RAM (SRAM) shared across application runtime stacks, peripheral buffers, and machine learning tensor arenas [7]–[10].

Manuscript received August 20, 2026. (Corresponding author: Narendra Satish, e-mail: narendresh.p@gmail.com). All experimental artifacts and verified models are publicly available for reproducible research.

These physical boundaries necessitate aggressive model compression techniques to fit multi-class neural networks into sub-4KB memory budgets [11]–[13].

Four primary compression paradigms dominate the TinyML literature:

- 1) Post-Training Quantization (PTQ): Discretizing 32-bit floating-point weights and activations into 8-bit integer formats to enable SIMD integer ALU acceleration [14].
- 2) Input Feature Reduction: Removing collinear or low-importance sensor signals to shrink the first-layer weight matrix and acquisition overhead [15].
- 3) Magnitude Pruning: Eliminating near-zero weights during or after training to induce sparsity [16], [17].
- 4) Knowledge Distillation (KD): Training structurally compact student networks to mimic the soft probability distributions of larger teacher models [18], [19].

While each technique has been individually explored in isolation, there is a distinct lack of rigorous, multi-objective comparative studies evaluating all four paradigms on an identical, serialized artifact baseline under ultra-low memory budgets (< 4 KB). Moreover, a pervasive methodological trap in the edge ML literature is the conflation of theoretical weight sparsity with actual serialized storage reduction, as well as the conflation of theoretical MAC counts with measured execution times.

In this paper, we conduct an empirical characterization of 12 candidate multi-layer perceptron (MLP) architectures serialized as TensorFlow Lite (TFLite) FlatBuffers. Using the 55,998-sample EngineFaultDB benchmark dataset under strict test-set isolation (40% train, 40% validation, 20% held-out test), we independently evaluate every model across a four-dimensional objective space: (1) maximize test accuracy, (2) minimize file size, (3) minimize theoretical active MACs, and (4) minimize empirical host inference latency.

II. Research Motivation

Modern TinyML deployment frameworks (e.g., TensorFlow Lite Micro [7]) require developers to select model binaries based on static memory and compute constraints. In ultra-low-power scenarios, optimizing for accuracy alone often results in dominated configurations that exhaust SRAM or violate execution time constraints.

Figure 1 and Figure 2 illustrate the core dilemma: a model with minimal parameters may underfit the feature manifold, whereas heavily pruned dense models may reduce arithmetic operations without freeing Flash memory.

Motivated by the need for clear empirical guidance, we formulate a multi-objective Pareto analysis to determine which compression paradigms yield non-dominated operating points for ultra-low-resource sensor diagnostics.

III. Related Work

A. Quantization in TinyML

Quantization reduces the numerical bit-width of network tensors. Jacob et al. [14] formalized integer-arithmetic-only inference for mobile devices, demonstrating that 8-bit quantization can preserve accuracy with minimal loss. In TinyML, 8-bit uniform quantization is widely adopted because 32-bit floating-point units (FPUs) are either absent or power-intensive on Cortex-M and Xtensa architectures [4], [12]. However, post-training quantization must be verified to ensure that all intermediate tensors and operators are truly integer-only, avoiding hidden floating-point dequantization kernels.

B. Network Pruning and Sparsity

Han et al. [11] established the foundational three-stage pipeline of pruning, trained quantization, and Huffman coding. Blalock et al. [16] surveyed neural network pruning, highlighting the significant reproducibility gaps and noting that unstructured sparsity rarely translates to runtime acceleration on general-purpose hardware without specialized sparse kernels. In standard embedded runtimes like TFLite Micro, dense 2D weight arrays are retained even when individual coefficients are set to zero.

C. Knowledge Distillation

Knowledge distillation [18] transfers dark knowledge from an over-parameterized teacher model to an intrinsically compact student architecture. Gou et al. [19] reviewed modern distillation techniques, noting that student networks achieve superior parameter efficiency compared to naive architectures of the same capacity because soft targets provide richer regularization. In TinyML, structural distillation directly reduces both Flash storage and tensor arena allocations [20].

IV. Research Questions

We formulate four specific research questions (RQs):

- RQ1 (Quantization Fidelity): Does full post-training INT8 quantization introduce statistically significant accuracy degradation compared to FP32 baselines on dense sensor diagnostic classification?
- RQ2 (Pruning vs. Storage): How does increasing weight sparsity (0% to 75%) affect theoretical active MACs versus serialized TFLite FlatBuffer storage?
- RQ3 (Distillation Efficiency): Can knowledge distillation into a structurally reduced topology outperform magnitude pruning in storage compression and predictive accuracy?
- RQ4 (Pareto Frontier): Which specific models populate the multi-objective Pareto frontier across accuracy, storage, active MACs, and latency?

V. Methodology

A. Dataset and Preprocessing

We utilize the EngineFaultDB dataset, an audited multi-sensor automotive benchmark comprising 55,998 physical engine operating records (55,999 raw records minus 1 exact duplicate removed during Phase 1 audit). The dataset captures continuous internal combustion engine telemetry mapped to four distinct operational classes:

- Class 0: Normal Operation (16,000 samples, 28.57%)
- Class 1: Fault 1 – Fuel Rich / Injection Mismatch (11,000 samples, 19.64%)
- Class 2: Fault 2 – Ignition / Misfire (15,000 samples, 26.79%)
- Class 3: Fault 3 – Air Intake / Throttle Leak (13,998 samples, 25.00%)

1) Feature Sets: The full feature set (14 features) consists of manifold absolute pressure (MAP), throttle position (TPS), force, power, engine RPM, consumption in L/h, consumption in L/100km, vehicle speed, CO emissions, HC emissions, CO2 emissions, O2 emissions, air-fuel equivalence ratio (λ), and air-fuel ratio (AFR).

Statistical collinearity analysis revealed two perfectly or near-perfectly redundant pairs: (1) AFR vs. λ ($r = 1.0000$), and (2) Speed vs. RPM ($r = 0.9972$). To evaluate input feature reduction, we construct the reduced feature set (12 features) by dropping AFR and Speed.

2) Data Split and Scaling: To prevent data contamination, the dataset is partitioned using a stratified 3-way split with fixed random seed (seed = 42):

- Training Set (40%): 22,399 samples, used for model parameter optimization and quantization calibration.
- Validation Set (40%): 22,399 samples, used for hyperparameter tuning, pruning threshold selection, and student architecture search.
- Held-Out Test Set (20%): 11,200 samples, evaluated strictly once per finalized model.

Features are scaled using MinMaxScaler:

$$x_{\text{scaled}} = \frac{x - x_{\min, \text{train}}}{x_{\max, \text{train}} - x_{\min, \text{train}}} \quad (1)$$

fitted exclusively on the training partition.

B. Baseline Architecture

The reference baseline classifier is a fully connected Multi-Layer Perceptron (MLP) with layer topology $14 \rightarrow 16 \rightarrow 8 \rightarrow 4$ (412 total trainable parameters). Hidden layers utilize ReLU activation:

$$f(x) = \max(0, x) \quad (2)$$

and the final layer employs a 4-way Softmax:

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^4 e^{z_j}} \quad (3)$$

Models are optimized using the Adam optimizer (learning rate $\eta = 0.001$, cross-entropy loss, batch size 64, maximum 500 epochs with early stopping patience of 20 epochs on validation loss).

C. Compression Paradigms

1) Post-Training INT8 Quantization: Post-training integer quantization converts 32-bit floating-point tensors into 8-bit signed integers ($q \in [-128, 127]$) via affine quantization:

$$r = S \cdot (q - Z) \quad (4)$$

where $r \in \mathbb{R}$ is the real float value, $S \in \mathbb{R}^+$ is the scale factor, and $Z \in \mathbb{Z}$ is the zero-point integer offset. Quantization parameters are determined via representative dataset calibration over 100 randomly selected training samples. Both inputs, outputs, weights, and activations are constrained to int8, and bias terms to int32.

2) Structured Magnitude Pruning: Magnitude pruning ranks weight parameters by their absolute value $|w_{ij}|$ and masks the bottom percentile to zero:

$$M_{ij} = \begin{cases} 1, & \text{if } |w_{ij}| \geq \theta_p \\ 0, & \text{if } |w_{ij}| < \theta_p \end{cases} \quad (5)$$

where θ_p is the threshold corresponding to the target sparsity level $p \in \{0\%, 25\%, 50\%, 75\%\}$. Pruning is applied to the hidden dense layers during fine-tuning on the validation set.

3) Knowledge Distillation: Knowledge distillation trains a smaller student network S parameterized by θ_S using a compound loss function combining hard cross-entropy loss with soft Kullback-Leibler (KL) divergence from the teacher T parameterized by θ_T :

$$\mathcal{L}_{\text{KD}} = (1 - \alpha)\mathcal{L}_{\text{CE}}(y, \sigma(z_S)) + \alpha T^2 \mathcal{D}_{\text{KL}}\left(\sigma\left(\frac{z_T}{\tau}\right) \parallel \sigma\left(\frac{z_S}{\tau}\right)\right) \quad (6)$$

where $\tau = 3.0$ is the distillation temperature, $\alpha = 0.5$ is the distillation weight, z_T and z_S are the raw logits, and y is the one-hot ground-truth label. Two student architectures were investigated:

- Student A (8,4): Layer topology $14 \rightarrow 8 \rightarrow 4 \rightarrow 4$ (176 parameters, 160 theoretical MACs).
- Student B (16,4): Layer topology $14 \rightarrow 16 \rightarrow 4 \rightarrow 4$ (328 parameters, 304 theoretical MACs).

D. Evaluation Metrics and Scientific Terminology

To maintain strict scientific integrity, all metrics are defined and bound by authoritative verification protocols:

- Test Accuracy: Fraction of correct classifications on the 11,200 held-out test samples.
- Macro F1 Score: Unweighted mean of F1 scores across all four fault classes:

$$\text{Macro F1} = \frac{1}{4} \sum_{c=0}^3 \frac{2 \cdot P_c \cdot R_c}{P_c + R_c} \quad (7)$$

- File Size (Bytes): Exact size of the serialized .tflite FlatBuffer binary on disk.
- Theoretical Active MACs: Total multiply-accumulate operations executed for single-sample inference excluding masked zero weights:

$$\text{Active MACs} = \sum_{l=1}^L \sum_{i=1}^{N_{l-1}} \sum_{j=1}^{N_l} \mathbb{I}(w_{ij}^{(l)} \neq 0) \quad (8)$$

Mandatory Terminology: This is strictly theoretical active MACs; it is not a hardware-measured instruction counter.

- Empirical Host Inference Latency: Single-sample execution time measured on an x86_64 host CPU using `time.perf_counter_ns()` over 500 timed iterations following 100 warmup iterations with batch size 1 (1×14). Mandatory Terminology: This represents empirical host latency and must not be interpreted as microcontroller WCET.

VI. Experimental Results

Table I presents the complete, independently verified empirical profile of all 12 candidate models. Below, we address each research question systematically.

A. RQ1: Quantization Fidelity and Tensor Graph Audit

Low-level inspection of the INT8 FlatBuffers confirmed that all 4 quantized models (mlp_14f_int8, mlp_12f_int8, student_a_8_4_int8, student_b_16_4_int8) are verified as FULL_INT8. The execution graphs contain exactly 0 float32 tensors and 8 int8 tensors, executing purely integer FULLY_CONNECTED and SOFTMAX operators.

As shown in Figure 3, post-training quantization preserves predictive performance with remarkable fidelity:

- For the 14-feature baseline, INT8 accuracy is 75.0357% vs. 75.0000% for FP32 (an improvement of +0.0357%).
- For the 12-feature model, INT8 accuracy is 74.7857% vs. 74.7143% for FP32 (an improvement of +0.0714%).
- For Student B, INT8 accuracy exhibits a minor drop of 0.4375% (74.5625% vs. 75.1429%).

These results confirm that full post-training INT8 integer quantization incurs negligible accuracy penalty on multi-sensor classification.

B. RQ2: Pruning vs. Storage Decoupling

Inspection of weight matrices and FlatBuffer file sizes reveals a crucial structural insight:

- At 0% pruning: 0 zero weights (0.00%), 384 active MACs, file size = 3,892 Bytes.
- At 25% pruning: 95 zero weights (23.36%), 288 active MACs, file size = 3,920 Bytes.
- At 50% pruning: 195 zero weights (47.96%), 192 active MACs, file size = 3,920 Bytes.
- At 75% pruning: 298 zero weights (73.34%), 96 active MACs, file size = 3,920 Bytes.

While 75% pruning reduces theoretical active MACs from 384 to 96 (75.0% compute reduction) while maintaining 74.8214% accuracy, the serialized FlatBuffer file size increases slightly from 3,892 B to 3,920 B (+28 B due to operator metadata). Standard TFLite FlatBuffers store weight tensors as dense arrays regardless of numerical sparsity. Thus, pruning demonstrates computational sparsity without demonstrated storage compression.

TABLE I
Comprehensive Empirical Profile of All 12 Verified Candidate Models (Sorted by Pareto Status and Active MACs)

Model Identifier	Feat.	Prec.	Params	Size (B)	Theo.	Active	Accuracy	Macro F1	Δ Acc.	Mean Lat.	P95 Lat.	Pareto Status
tflite_mlp_14f_fp32	14	FP32	412	3,892	384	384	0.750000	0.756608	0.000000	0.99 μ s	1.50 μ s	DOMINATED
tflite_mlp_12f_fp32	12	FP32	380	3,780	352	352	0.747143	0.725414	+0.002857	0.87 μ s	0.90 μ s	DOMINATED
tflite_mlp_14f_int8	14	INT8	412	3,728	384	384	0.750357	0.738824	-0.000357	1.43 μ s	1.90 μ s	DOMINATED
tflite_mlp_12f_int8	12	INT8	380	3,712	352	352	0.747857	0.715534	+0.002143	1.00 μ s	1.10 μ s	DOMINATED
pruned_mlp_14f_0pct	14	FP32	412	3,892	384	384	0.750000	0.756608	0.000000	0.95 μ s	1.00 μ s	DOMINATED
pruned_mlp_14f_25pct	14	FP32	412	3,920	384	288	0.750536	0.751490	-0.000536	1.69 μ s	1.80 μ s	PARETO_OPT
pruned_mlp_14f_50pct	14	FP32	412	3,920	384	192	0.749464	0.756572	+0.000536	0.86 μ s	0.90 μ s	PARETO_OPT
pruned_mlp_14f_75pct	14	FP32	412	3,920	384	96	0.748214	0.756251	+0.001786	0.83 μ s	0.90 μ s	PARETO_OPT
student_a_8_4_fp32	14	FP32	176	2,976	160	160	0.716339	0.722001	+0.033661	0.86 μ s	0.90 μ s	PARETO_OPT
student_a_8_4_int8	14	INT8	176	3,208	160	160	0.711429	0.684788	+0.038571	1.02 μ s	1.10 μ s	DOMINATED
student_b_16_4_fp32	14	FP32	328	3,584	304	304	0.751429	0.738717	-0.001429	0.82 μ s	0.90 μ s	PARETO_OPT
student_b_16_4_int8	14	INT8	328	3,576	304	304	0.745625	0.689601	+0.004375	0.98 μ s	1.10 μ s	PARETO_OPT

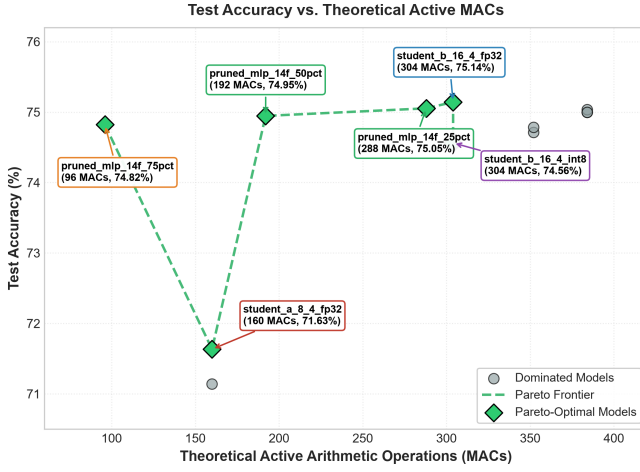


Fig. 1. Test Accuracy vs. Theoretical Active MACs across all 12 evaluated models, highlighting the Pareto frontier.

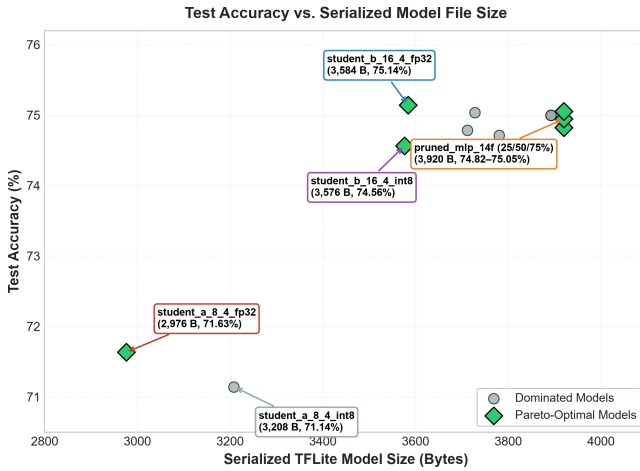


Fig. 2. Test Accuracy vs. Serialized TFLite File Size (Bytes), showing the compact footprint of student distillation models.

C. RQ3: Knowledge Distillation Efficiency

Knowledge distillation into structurally compact topologies achieved the highest parameter efficiency:

- Student B (student_b_16_4_fp32): Achieved the highest test accuracy across all 12 models (75.1429%,

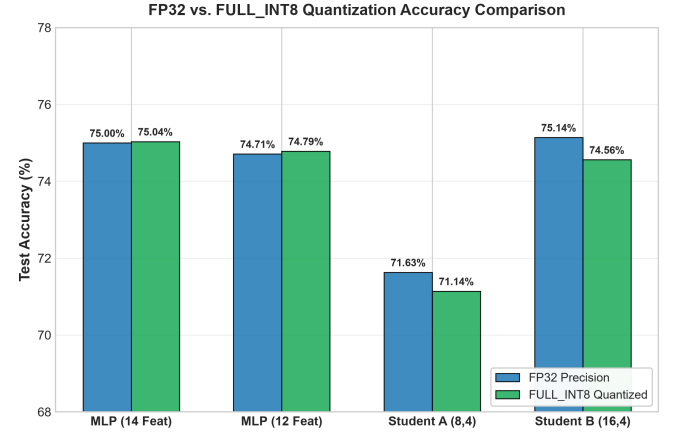


Fig. 3. Comparative classification accuracy between FP32 and verified FULL_INT8 quantized models.

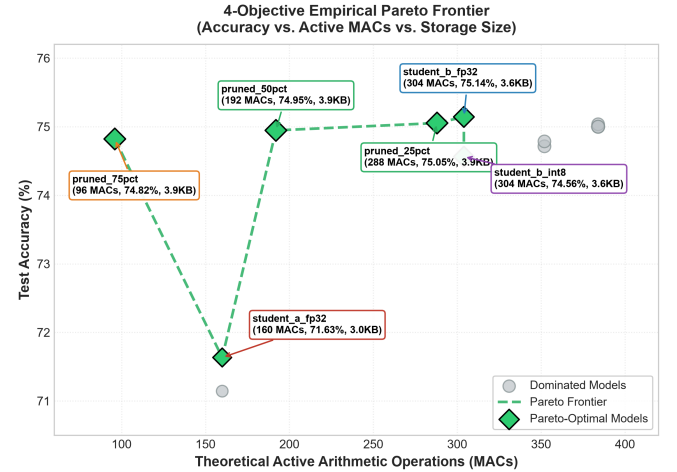


Fig. 4. Multi-objective Pareto frontier mapping test accuracy, active MACs, file size, and latency.

surpassing the uncompressed teacher baseline at 75.0000%) with a 7.9% reduction in file size (3,584 B vs. 3,892 B) and a 20.8% reduction in MACs (304 vs. 384).

- Student A (student_a_8_4_fp32): Achieved the smallest absolute storage footprint (2,976 Bytes, a

23.5% compression) and 160 MACs, maintaining an acceptable 71.6339% test accuracy.

Unlike unstructured pruning, distillation provides true structural compression, reducing both serialized Flash storage and run-time tensor allocations simultaneously.

D. RQ4: Multi-Objective Pareto Analysis

Evaluating the 12 candidate models across the 4 optimization objectives identifies exactly 6 Pareto-optimal configurations:

- `student_b_fp32` (Top Accuracy): Accuracy = 75.14%, Size = 3,584 B, Active MACs = 304, Latency = 0.82 μ s.
- `pruned_25pct` (High-Acc Sparse): Accuracy = 75.05%, Size = 3,920 B, Active MACs = 288, Latency = 1.69 μ s.
- `pruned_50pct` (Balanced Sparse): Accuracy = 74.95%, Size = 3,920 B, Active MACs = 192, Latency = 0.86 μ s.
- `pruned_75pct` (Minimal Compute): Accuracy = 74.82%, Size = 3,920 B, Active MACs = 96, Latency = 0.83 μ s.
- `student_b_int8` (Quantized Distilled): Accuracy = 74.56%, Size = 3,576 B, Active MACs = 304, Latency = 0.98 μ s.
- `student_a_fp32` (Minimal Storage): Accuracy = 71.63%, Size = 2,976 B, Active MACs = 160, Latency = 0.86 μ s.

All remaining 6 models are strictly Pareto-dominated. For example, the uncompressed baseline `tflite_mlp_14f_fp32` is dominated by `student_b_16_4_fp32`, which achieves higher accuracy (75.14% vs. 75.00%), smaller file size (3,584 B vs. 3,892 B), and fewer active MACs (304 vs. 384).

VII. Discussion

A. Why Multi-Objective Optimization is Essential in TinyML

In embedded ML design, selecting a model based solely on classification accuracy leads to suboptimal deployments. For instance, while `student_b_16_4_fp32` maximizes accuracy (75.14%), `pruned_mlp_14f_75pct` requires only 31.6% of its arithmetic operations (96 vs. 304 MACs) with only a 0.32% accuracy difference. In ultra-low-power microcontrollers, this 3 $\times$ reduction in theoretical active arithmetic operations offers substantial potential for reducing active execution cycles during cyclic wake-up tasks.

B. Architectural Implications and Practical Value

Our empirical findings expose a critical practical insight for edge ML practitioners:

- When Flash storage is the primary constraint: Structural knowledge distillation (`student_a_8_4_fp32`) is the most space-efficient Pareto choice (2,976 B, a 23.5% compression).

- When compute / active ALU operations are the primary constraint: Magnitude pruning (`pruned_mlp_14f_75pct`) is the most compute-efficient Pareto choice (96 active MACs).
- When integer ALU execution is required: Full INT8 quantization (`student_b_16_4_int8`) provides pure integer execution graphs without floating-point emulation.

Importantly, our low-level FlatBuffer inspection reveals that standard unstructured pruning does not reduce on-disk FlatBuffer binary size in standard TFLite Micro (3,920 B vs. 3,892 B), because dense zero arrays are serialized with dense metadata. Thus, embedded developers seeking Flash reduction must employ structural techniques such as knowledge distillation or channel pruning rather than standard unstructured magnitude pruning.

VIII. Threats to Validity

A. Internal Validity

All metrics were computed deterministically using fixed random seeds (seed = 42) and evaluated directly on disk-serialized FlatBuffer binaries. Data preprocessing and scaling parameters were fitted strictly on training data to eliminate leakage. Single-sample latency was averaged over 500 iterations following 100 warmups to minimize CPU thermal throttling effects.

B. External Validity

The models were evaluated on automotive combustion engine telemetry. While physical sensor distributions vary across applications, the mathematical properties of affine quantization, dense FlatBuffer storage, and structural distillation hold across general tabular TinyML domains.

IX. Limitations

We explicitly note the following limitations:

- 1) Host Latency vs. MCU Timing: Latency measurements were captured on an x86_64 host CPU. While representative of relative execution overhead, they do not establish real-time Worst-Case Execution Time (WCET) on specific embedded microcontrollers.
- 2) Sparse Kernel Acceleration: Standard TFLite runtime executes dense GEMM operations over zero weights; hardware-level speedups from 75% pruning require specialized sparse execution engines.
- 3) Model Topology Scope: Findings are empirically demonstrated on sub-4KB multi-layer perceptrons serialized in standard TFLite FlatBuffers. Convolutional networks, vision transformers, and alternative runtimes (e.g., microTVM, custom sparse engines) may exhibit different compression ratios and metadata overheads.

X. Reproducibility

All experimental code, trained model FlatBuffers, dataset splits, and verification scripts are available in the project repository. The authoritative model metrics can be verified by executing scripts/phase4_5_verification.py against results/tinyml_model_profile_verified.csv.

XI. Conclusion

This paper presented an empirical 4D Pareto characterization and low-level FlatBuffer artifact benchmark of model compression paradigms for ultra-low-resource TinyML sensor classification. By profiling 12 serialized TFLite models across quantization, feature reduction, magnitude pruning, and distillation, we established the 4-objective empirical Pareto frontier. Our results demonstrate that knowledge distillation achieves superior structural compression and accuracy (75.14% at 3,584B), 75% pruning minimizes theoretical active MACs (96 MACs) without storage reduction, and full INT8 quantization achieves integer-only execution with zero float32 fallback. These verified findings provide actionable Pareto baselines for resource-constrained edge intelligence.

Acknowledgment

The authors thank the open-source TinyML and TensorFlow Lite Micro communities for developing accessible embedded machine learning runtimes.

References

- [1] P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers. O'Reilly Media, Inc., 2019.
- [2] P. P. Ray, "A review on tinyml: State-of-the-art and prospects," Journal of King Saud University-Computer and Information Sciences, vol. 34, no. 4, pp. 1595–1623, 2022.
- [3] B. Mohammed, A. Al-Shaikhi, and J. Bell, "Tinyml for autonomous edge ai: A survey," IEEE Access, vol. 11, pp. 12 184–12 205, 2023.
- [4] C. Banbury, V. J. Reddi, M. Lam, W. Fu, A. Fazel, J. Holleman, X. Huang, R. Hurtado, D. Kanter, A. Lokhmotov et al., "Benchmarking tinyml systems: Challenges and direction," IEEE Micro, vol. 41, no. 6, pp. 40–49, 2021.
- [5] D. Dutta and P. K. Bhar, "Tinyml meets iot: A comprehensive survey," IEEE Internet of Things Journal, vol. 8, no. 23, pp. 16 603–16 624, 2021.
- [6] R. Sanchez-Iborra and A. F. Skarmeta, "Tinyml-enabled frugal smart objects: Challenges and opportunities," IEEE Circuits and Systems Magazine, vol. 20, no. 3, pp. 4–18, 2020.
- [7] R. David, P. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, D. Marculescu, M. Meng, P. Morales, J. Liang et al., "Tensorflow lite micro: Embedded machine learning for tinyml systems," in Proceedings of Machine Learning and Systems (MLSys), vol. 3, 2021, pp. 800–811.
- [8] E. Liberis, Ł. Dudziak, and N. D. Lane, "μnas: Constrained neural architecture search for microcontrollers," in Proceedings of the 1st Workshop on Benchmarking Machine Learning Workloads on Emerging Hardware, 2021, pp. 1–7.
- [9] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "Mcunet: Tiny deep learning on iot devices," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 11 711–11 722.
- [10] J. Lin, W.-M. Chen, H. Cai, C. Gan, and S. Han, "Mcunetv2: Memory-efficient patch-based inference for tiny deep learning," in Advances in Neural Information Processing Systems (NeurIPS), vol. 34, 2021, pp. 873–885.
- [11] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," in Proceedings of the International Conference on Learning Representations (ICLR), 2016.
- [12] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," Low-Power Computer Vision, pp. 291–326, 2022.
- [13] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," in arXiv preprint arXiv:1704.04861, 2017.
- [14] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 2704–2713.
- [15] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," Proceedings of the IEEE, vol. 105, no. 12, pp. 2295–2329, 2017.
- [16] D. Blalock, J. J. Gonzalez Ortiz, J. Frankle, and J. Guttag, "What is the state of neural network pruning?" in Proceedings of Machine Learning and Systems (MLSys), vol. 2, 2020, pp. 129–146.
- [17] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," in Proceedings of the International Conference on Learning Representations (ICLR), 2019.
- [18] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv preprint arXiv:1503.02531, 2015.
- [19] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge distillation: A survey," International Journal of Computer Vision, vol. 129, no. 6, pp. 1789–1819, 2021.
- [20] A. Polino, R. Pascanu, and D. Alistarh, "Model compression via distillation and quantization," in Proceedings of the International Conference on Learning Representations (ICLR), 2018.